Showing and explaining the C2 results

> Printable edition. Use the Contents links or PDF bookmarks to navigate. Long code lines wrap for printing; copy executable commands from the Markdown guide. Source-file links refer to the local repository; keep the three PDFs together for related-guide links.

Each local file link is followed by its full path as plain text. A final `:number` identifies the source line; in VS Code, open the file with `Ctrl+P` and jump to that line with `Ctrl+G`.

Use this alongside the component study guide (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/COMPONENT_STUDY_GUIDE.md`) and metrics reference (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/METRICS_REFERENCE.md`). All commands below inspect completed results, except the clearly labelled local web-server command. None starts dataset generation, training, calibration or test evaluation. Work from `D:/VSCode/GitHub/Synth-Inversion-RP`.

Contents

Choose a route or jump to a specific stage. The **Outline** panel in VS Code shows these headings and their smaller steps.

- VS Code and terminal route
- Browser demo route
- Short demonstration
- Follow-up questions

Stage	Responsibility	Jump to
Dataset generation	Shared generation pipeline	Stop 1: Shared dataset generation
Audio conditioning	Shared loader used by C2	Stop 2A: Audio conditioning with the shared utility
Train/calibration/validation/test split	C2 data preparation	Stop 2B: C2 dataset splitting
Log-mel features and normalization	C2 preprocessing	Stop 3: C2 feature preprocessing and normalization
Single CNN training and checkpoints	C2 model training	Stop 4: Single CNN training
Ensemble training and uncertainty	C2 model training	Stop 5: Ensemble training and uncertainty U
One real predicted clip	C2 saved outputs	Stop 6: Trace one saved prediction
Reconstruction R and sensitivity S	C2 forward diagnostics	Stop 7A: Forward diagnostics with R and S
Correctness labels, fitting and q	C2 reliability calibration	Stop 7B: Reliability calibration and q
Metrics, intervals and robustness	C2 evaluation/validation	Stop 8: Evaluation and validation

Folder name is not the same as processing stage: `generation/corpus_audio.py` supplies the shared audio loader. C2's `v4/data.py` creates its experimental splits. Displaying `protocol.json` only shows the saved record.

Browser route shortcuts

- B1. Open the local demo
- B2. Demonstrate audio, parameters and diagnostics
- B3. Compare reliability models and examples

- B4. Show validation results and method

Before starting

Keep this sequence in mind:

```
Shared data: choose settings -> render audio -> record actual labels -> audit
C2 fitting: freeze splits -> log-mel features -> single CNN -> three probabilistic CNNs
C2 reliability: freeze predictions -> render R/S -> fit calibrators -> compare ablations
Show results: original/predicted audio -> settings -> U/S/R/q -> population evidence
```

The short opening is:

> I am testing whether forward synthesizer measurements can improve the reliability of individual parameter predictions. The original prediction stays fixed, and every reliability method judges that same prediction.

For every result, say **which examples, how many, which units and which metric**. Do not switch between the historical eight-control pilot and current 13-control v4.

Route A: use VS Code and its terminal

Open the repository folder. `Ctrl+P` opens a file by path; `Ctrl+G` jumps to a line. For this guide, use `Ctrl+Shift+V` to open Markdown preview. The absolute file links also identify exactly which source/result is meant. The optional `code --goto` commands work when the VS Code command-line launcher is on PATH; otherwise use `Ctrl+P` and `Ctrl+G` with the same path and line.

Stop 1: Shared dataset generation

1A. Configuration, sampling and rendering

Open these in order:

1. bend_sustain_config.py:28 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/bend_sustain_config.py:28`): the current parameter schema and inheritance.
2. restricted_config.py:22 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/restricted_config.py:22`): fixed note, rate, timing and generation seed.
3. create_sobol_samples:206 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/scripts/05_generate_pilot_dataset.py:206`): structured sampling.
4. apply_sampled_parameters:354 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/scripts/05_generate_pilot_dataset.py:354`): set knobs and record actual readbacks.
5. render_one:225 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/generate_corpus.py:225`): fresh synth and one complete render.

Say: "The knobs and fixed recipe define my experimental domain. Each sound has known generating settings because we controlled the synth. Sobol chooses settings; the engine readback supplies the actual label."

1B. Dataset integrity checks

Show the completed full-corpus audit without rerunning the full disk scan:

```
Get-Content -Raw .\data\processed\team_handoff_v4_131072\verification.json |
    ConvertFrom-Json |
    Select-Object verified_rows, unique_raw_audio_paths, uncheckpointed_wavs,
```

```
        raw_and_processed_hashes_verified, manifest_csv_equals_parquet_equals_checkpoints,
        quality_flags |
    Format-List
```

Expected: 131,072 rows/unique raw paths, zero uncheckpointed WAVs, matching waveform/manifest checks and zero conditioned QA flags. This displays an existing verification record; it is not a fresh full-corpus verification.

1C. Inspect one generated record

To inspect a real manifest header and the first row:

```
Get-Content .\data\raw\corpus_v4\manifest.csv -TotalCount 2
```

It is wide; use it to point out `sample_id`, `raw_audio_path`, `*_sample_01`, `*_normalized`, `*_raw`, `*_display` and hashes, not to read every value aloud. Parquet files are binary tables; VS Code's plain text view is not a table reader.

Stop 2A: Audio conditioning with the shared utility

Open corpus_audio.load_audio:48 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/generation/corpus_audio.py:48`).

This function reads raw audio and applies the fixed DC policy. It lives in the shared `generation/` folder, and C2 imports it when loading audio. It does not generate new clips or decide which clips belong to training or test.

Stop 2B: C2 dataset splitting

Open data.assignments:72 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/data.py:72`). This C2 function creates the train/calibration/validation/test assignments. Show protocol.json (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/protocol_v1/protocol.json`).

The command below displays the saved C2 protocol; it does not create or change a split.

```
$c2Protocol = Get-Content -Raw .\data\processed\c2_v4\protocol_v1\protocol.json | ConvertFrom-Json
$c2Protocol | Select-Object seed, split_counts, validation_roles,
    development_prefix_training_only, diagnostic_counts, selection_policy,
    delta_z, tau_z, test_policy | Format-List
```

Recap of audio conditioning and C2 splitting

Say: "I use a fixed DC rule once, then preserve different roles for training, reliability fitting, development selection and final testing. The previously inspected 1,024 v4 examples are training-only. Subsets were selected before looking at errors."

State the counts: 98,304 train; 16,384 calibration pool; 4,096 inverse-validation; 4,096 reliability-validation; 8,192 reserved test. Reliability uses 2,048 and 512 predetermined examples from its two pools. No model/reliability test evaluation has been performed. Whole-corpus file/format integrity checks are a separate data-QA step.

Stop 3: C2 feature preprocessing and normalization

Open extract_log_mel:323 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/pipeline.py:323`), features.training_statistics:117 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/features.py:117`) and a saved spectrogram under `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/baseline_full_v1/spectrogram_examples/`. Click a PNG in the Explorer to use VS Code's image preview.

```
.\venv\Scripts\python.exe -B -c 'import numpy as np;
    a=np.load("data/processed/c2_v4/features_full_v1/shard_0000.npy", mmap_mode="r"); print("Shard:", a.shape,
```

```
    a.dtype); print("One clip:", a[0].shape)'
```

Expected: shard `(512, 64, 259)`, float32; one clip `(64, 259)`. Say: "The network gets the numeric time-frequency array, not this PNG. The same training-derived mean and standard deviation are used for later inputs."

Stop 4: Single CNN training

4A. Training code and learning curves

Open ParameterCNN:82 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/pipeline.py:82`) and baseline.train:99 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/baseline.py:99`). Explain convolution/pooling, sigmoid outputs, MSE, backward pass and Adam using study-guide section 9. Open the learning-curve PNG beside the full baseline weights.

4B. Parameter-recovery results on inverse validation

```
Get-Content .\data\processed\c2_v4\baseline_full_v1\summary.json
Import-Csv .\data\processed\c2_v4\baseline_full_v1\validation_metrics.csv |
    Where-Object coordinate -eq research_z |
    Format-Table parameter, count, mae, rmse, r_squared -AutoSize
```

Say: "Epoch 53 was selected on inverse-validation MSE. The checkpoint reload reproduced the exported predictions. My macro z MAE is .126112 on 4,096 clips." Point out strong envelope/cutoff results and weaker bend/filter-envelope results.

4C. Saved checkpoint metadata

A `.pt` checkpoint is a binary PyTorch file, not readable source. Inspect its metadata:

```
.\venv\Scripts\python.exe -B -c 'import torch;
    p=torch.load("data/processed/c2_v4/baseline_full_v1/best_model.pt", map_location="cpu",
    weights_only=True); print("Best epoch:",p["epoch"]); print("Output
    count:",p["model"]["regressor.3.bias"].shape[0])'
```

Expected: 53 and 13. This loads existing weights; it neither trains nor evaluates test.

Stop 5: Ensemble training and uncertainty U

Open GaussianCNN:29 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/ensemble.py:29`) and combine:55 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/ensemble.py:55`).

```
$c2Ensemble = Get-Content -Raw .\data\processed\c2_v4\ensemble_full_v1\summary.json | ConvertFrom-Json
$c2Ensemble.members | Format-Table member, seed, best_epoch, checkpoint_reload_exact
$c2Ensemble | Select-Object macro_validation_z_mae, validation_z_mse, test_evaluated
```

Say: "The model averages three parameter predictions and combines within-model variance with between-model disagreement. U is variance, not a percentage of correctness. Ensemble macro MAE is .127991, so point accuracy did not improve overall."

The complete ensemble's checkpoints are:

- member_00/best_model.pt (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/ensemble_full_v1/member_00/best_model.pt`)
- member_01/best_model.pt (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/ensemble_full_v1/member_01/best_model.pt`)

- member_02/best_model.pt (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/ensemble_full_v1/member_02/best_model.pt`)

Stop 6: Trace one saved prediction

6A. Predicted settings and diagnostics

Use the saved viewer bundle as readable JSON even without using the browser:

```
$c2Demo = Get-Content -Raw .\data\processed\c2_v4\viewer_v1\data.json | ConvertFrom-Json
$c2Example = $c2Demo.examples[0]
$c2Example | Select-Object sample_id, diagnostic_rank, R, roundtrip_exact, predictions_unchanged
$c2Example.parameters | Select-Object name, actual_z, predicted_z, error_z, U, S,
    @{Name='q_full'; Expression={$_.q.'g(U,S,R)'}} | Format-Table -AutoSize
```

Expected ID: `restricted_bend_sustain_v4_63764`; R about .098242; roundtrip and prediction-preservation true. This is one illustrative validation example, not a replacement for aggregate results.

6B. Listen to target and reconstruction

To play its PCM16 WAVs from the Windows PowerShell terminal inside VS Code:

```
$c2AudioFolder = Join-Path (Get-Location)
    'data\processed\c2_v4\viewer_v1\examples\restricted_bend_sustain_v4_63764'
$c2Player = [System.Media.SoundPlayer]::new((Join-Path $c2AudioFolder 'target.wav'))
$c2Player.PlaySync()
$c2Player = [System.Media.SoundPlayer]::new((Join-Path $c2AudioFolder 'predicted.wav'))
$c2Player.PlaySync()
```

Use a comfortable volume. `PlaySync()` holds this terminal until the three-second file finishes; it does not start a model. This Windows audio type and WAV loading were checked without making sound during documentation QA. If audio output is unavailable in the terminal environment, use Route B's browser players.

6C. Inspect one parameter in detail

Show cutoff in detail:

```
$c2Cutoff = $c2Example.parameters | Where-Object name -eq filter_cutoff
$c2Cutoff | Select-Object actual_z, predicted_z, error_z, vital_normalized,
    minus_z, plus_z, actual_step_z, U, S, q | Format-List
$c2Cutoff.S * $c2Cutoff.actual_step_z
```

The last expression recovers minus-versus-plus audio distance, about .070776. Explain: z prediction .371036 maps to normalized cutoff .504070; error .001970 is inside .05; S=1.769388 is local activity; U=.001301 is variance; full q=.893669 is estimated correctness. The same clip's drive prediction is outside tolerance despite its fairly low global R.

Stop 7A: Forward diagnostics with R and S

Open these functions in sequence:

1. Renderer.render:90 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/diagnostics.py:90`): map settings and generate the predicted sound.
2. feature_distance:44 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/diagnostics.py:44`): the actual audio distance.
3. perturbation_pair:62 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/diagnostics.py:62`): central/boundary handling.

4. diagnostic_batch.measure:31 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/diagnostic_batch.py:31`): isolation, actual span, frozen prediction and audits.

Stop 7B: Reliability calibration and q

1. calibration.labels:39 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/calibration.py:39`): the within-tolerance event.
2. calibration.fit:46 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/calibration.py:46`): the small logistic calibrator.
3. calibration.run:165 (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/v4/calibration.py:165`): same predictions, feature ablations, validation C selection.

Recap of forward diagnostics and reliability calibration

Say: "I render at the predicted settings, not at the true settings. I move one parameter at a time and never keep a better perturbation. Separate data teaches the probability model; known error is a label, never an input feature."

Stop 8: Evaluation and validation

8A. Compare reliability methods and statistical intervals

```
Import-Csv .\data\processed\c2_v4\results_audit_v1\macro_scores.csv | Format-Table -AutoSize
Import-Csv .\data\processed\c2_v4\results_audit_v1\macro_paired_comparisons.csv |
    Format-Table variant, reference, group, point_difference, ci95_low, ci95_high -AutoSize
```

8B. Perturbation step-size validation

```
Get-Content -Raw .\data\processed\c2_v4\step_stability_v1\status.json
```

8C. Implementation verification

```
Get-Content -Raw .\data\processed\c2_v4\verification_validation_viewer_v1\verification.json |
    ConvertFrom-Json | Select-Object status, tests_run, skipped, failures, errors, test_set_evaluated
```

8D. Figures and interpretation

Open per_parameter_brier_differences.png (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/results_audit_v1/per_parameter_brier_differences.png`), matched_coverage_audit.png (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/results_audit_v1/matched_coverage_audit.png`) and step_size_rank_agreement.png (path: `D:/VSCode/GitHub/Synth-Inversion-RP/data/processed/c2_v4/step_stability_v1/step_size_rank_agreement.png`).

Say: "Full-model Brier is .115385 versus .116267 for U alone. The conditional comparison against U+R has an interval including zero, so S's extra benefit remains uncertain. Most sensitivity ranks are consistent across steps, with important exceptions. These are development results; the final test has not been evaluated."

Route B: use the browser demonstration

B1. Open the local demo

If the local page already loads, use that server. If it is not running, start this from the repository's PowerShell terminal:

```
.\venv\Scripts\python.exe -B -m http.server 8765 --bind 127.0.0.1 --directory data/processed/c2_v4/viewer_v1
```

Leave the terminal open. `Ctrl+C` stops this server. If the port belongs to another process, use another port instead of killing it. The complete launch/runbook is DEMO.md (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/DEMO.md`).

For a browser inside VS Code, open the Command Palette, choose **Simple Browser: Show**, and enter the URL. This is an optional viewing route; the existing local page has been tested in the Codex browser, not in every VS Code/browser build.

B2. Demonstrate audio, parameters and diagnostics

Action	Explain
Start on Audio and parameters, Example 1	"This is one of six saved examples, chosen by predefined validation order."
Play Original clip, then reconstruction	"The second clip is produced by Vital from predicted settings."
Expand log-mel comparison	"Both images use the same fixed scale. The CNN receives these numeric features."
Show original settings	"Synthetic labels let us check actual recovery. A new unknown recording would not have this reference."
Select filter cutoff	"The inspector changes only this parameter for its two example renders."
Play Lower setting and Higher setting	"Their difference divided by actual setting separation gives S."
Explain U, S, q and R	"U is variance; S is local activity; R is whole-clip mismatch; q estimates within-tolerance recovery."

B3. Compare reliability models and examples

Action	Explain
Change Reliability model to U only, then U+R, then U+S+R	"The probabilities change, but all methods judge the same settings and sound."
Pick another example	"The examples include both successes and failures. They were not chosen for high confidence."

B4. Show validation results and method

Action	Explain
Open Validation results	"These aggregate 512 cases, rather than judging the research by six sounds."
Show interval and matched-coverage figures	"I compare probability quality and which errors remain at the same retained fraction."
Open Method	"This is the implemented processing order and current domain."

The browser cannot show all dataset-generation/training source code. For those questions use the file links in this walkthrough or keep the relevant source tabs open beside it. Conversely, VS Code source alone does not demonstrate how a reconstructed clip sounds; the saved WAVs or browser players do that.

A short version if time is limited

1. Explain the question and 13-control domain.
2. Show actual-readback generation and frozen partitions.
3. Show log-mel input, CNN and single/ensemble results.
4. Play one real target/reconstruction and inspect cutoff plus a failed parameter.
5. Explain the freeze boundary, R/S, logistic q and ablations.
6. Show aggregate Brier/interval/coverage results and the remaining work.

Keep the full metrics reference (path: `D:/VSCode/GitHub/Synth-Inversion-RP/c2_estimation/METRICS_REFERENCE.md`) open for per-parameter questions. Do not spend the session scrolling through raw 131,072-row manifests or relaunching long experiments.

Useful answers when asked to go deeper

Why was this example selected? It is the first predefined reliability-validation diagnostic rank. The other five follow it in that same ordering.

How do you know reconstruction is implemented correctly? Known-parameter exact roundtrip, repeated-render equality, control isolation, checksummed source audio and frozen-prediction verification. Explain what each check establishes.

What would happen after uploading my own clip? The intended live service will apply supported input rules, predict settings, render a patch and compute U/S/R/q. That service/export interface remains pending. Arbitrary real-world recordings are not covered by today's calibration evidence.

Why don't all the knobs have good results? Some have weak or conditional audio effects and interactions; the estimator also has limited capacity and modelling error. The experiment measures those weaknesses and tests whether reliability information helps, rather than assuming the inverse problem is fully solvable.

What is left scientifically? Lock the final method and comparisons, evaluate the untouched test once, and later study whether a small group understands and finds the reliability display useful. No human-validation result is claimed today.